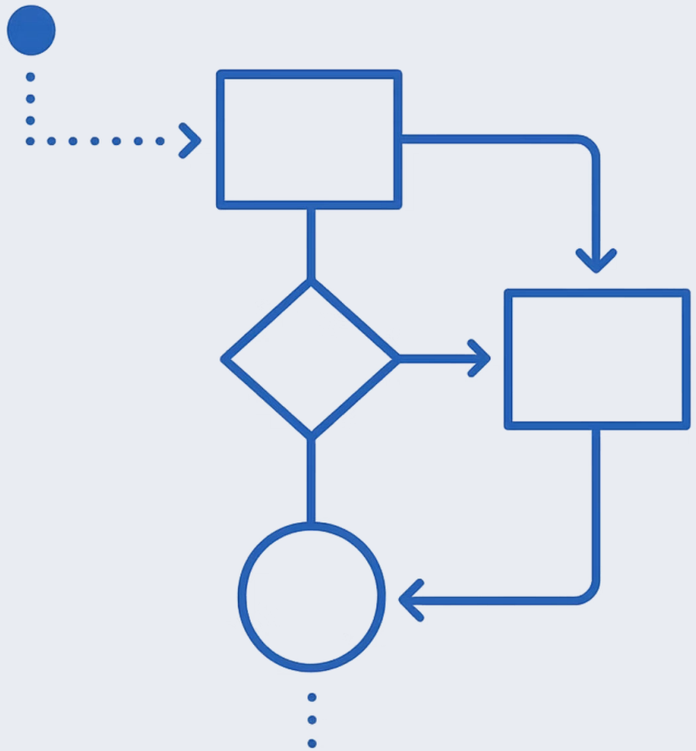
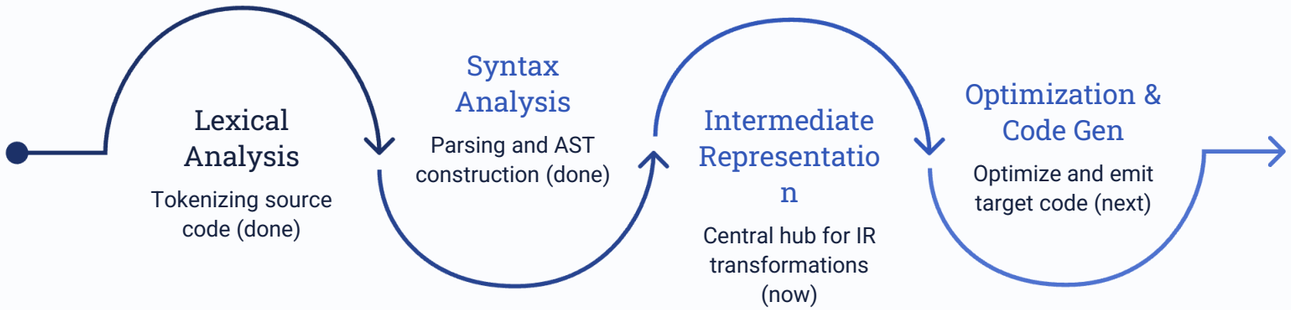


Three Address Code

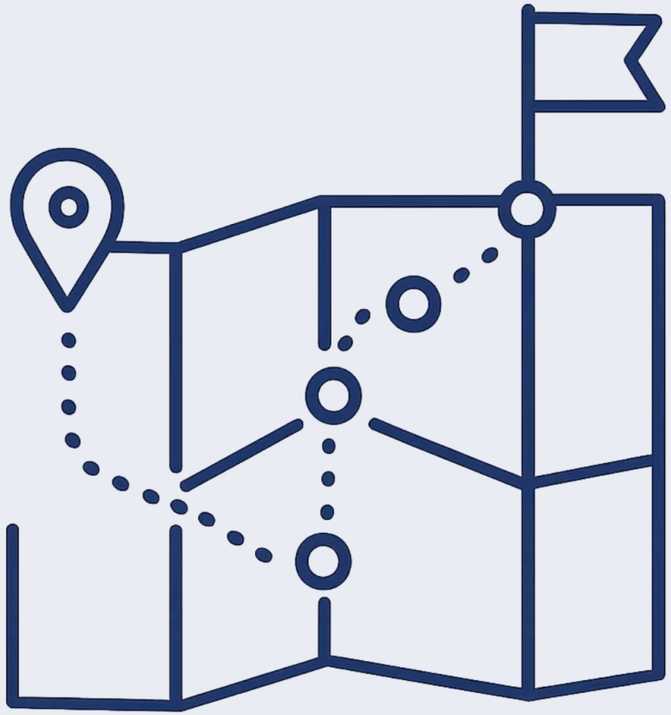
Intermediate code generation: types, representations, and syntax-directed translation into three address statements.

COMPILER DESIGN · CHAPTER 8

Where Are We in the Compiler Pipeline?



We've completed lexical analysis, syntax analysis, and syntax-directed translation (SDT). Now we move into IR — the central hub that enables optimization and target-independent code generation.



Why Do We Need IR?

Machine Independence

IR is not tied to any specific CPU or ISA — the same IR can target x86, ARM, or RISC-V.

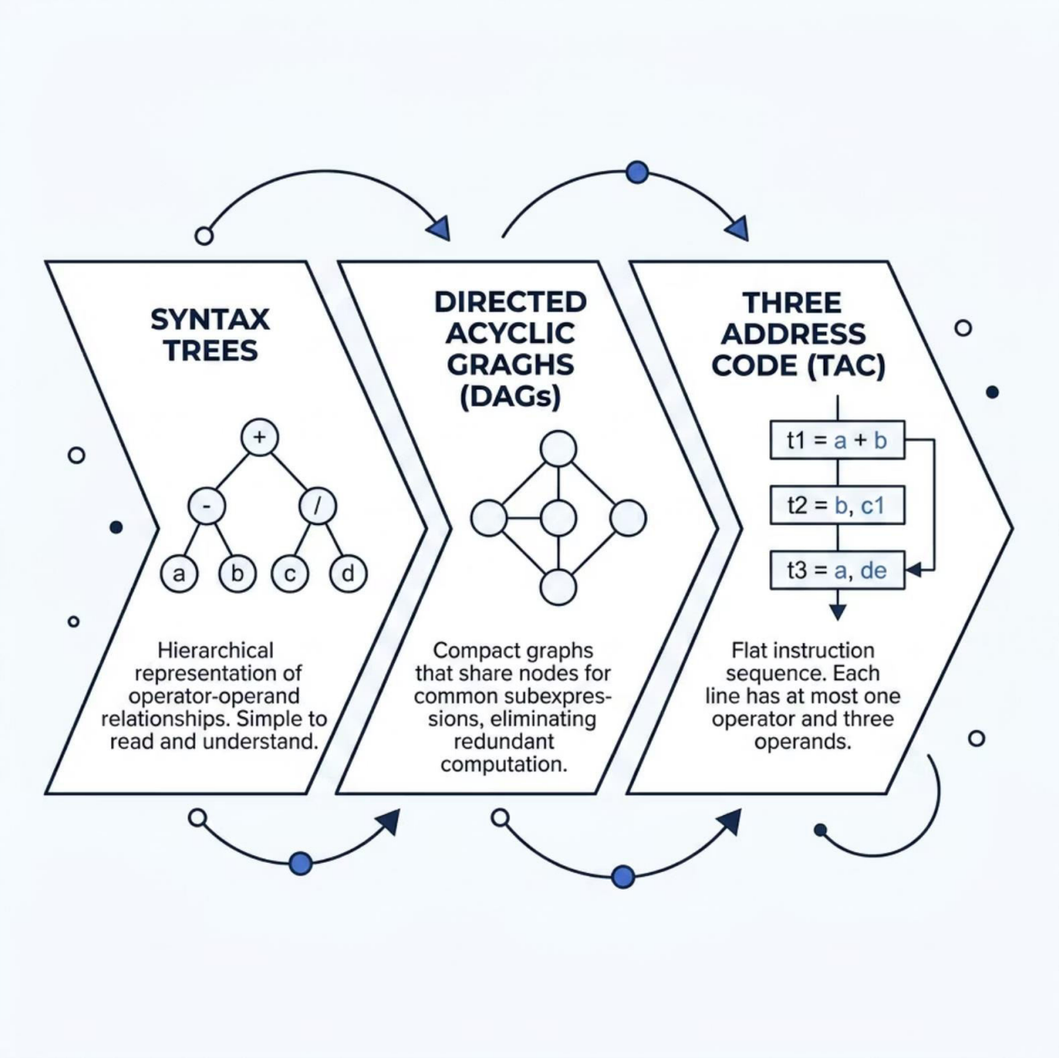
Optimization Ease

Optimizations like constant folding and dead code elimination are far simpler to apply at the IR level.

Simplified Code Generation

A well-structured IR reduces the complexity of mapping high-level constructs to low-level instructions.

Types of Intermediate Representation



Choosing the Right Form

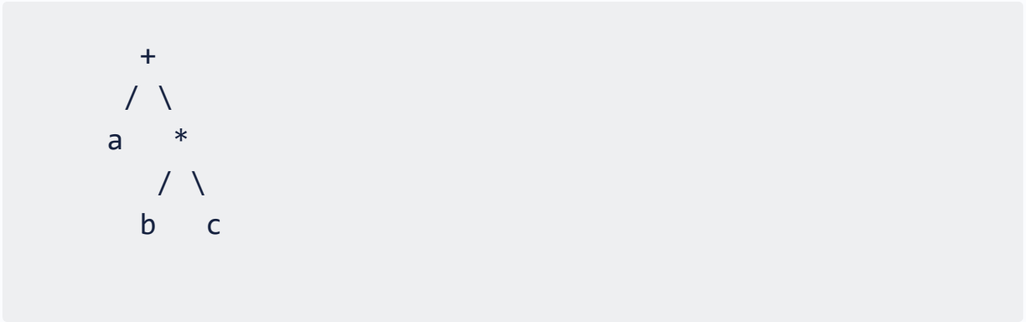
Each representation offers a different trade-off between **readability**, **compactness**, and **suitability for optimization**.

- **Syntax Trees** — closest to source; great for semantic analysis
- **DAGs** — compact; expose shared subexpressions automatically
- **TAC** — linearized; ideal for optimization passes and code generation

Syntax Trees vs. DAGs

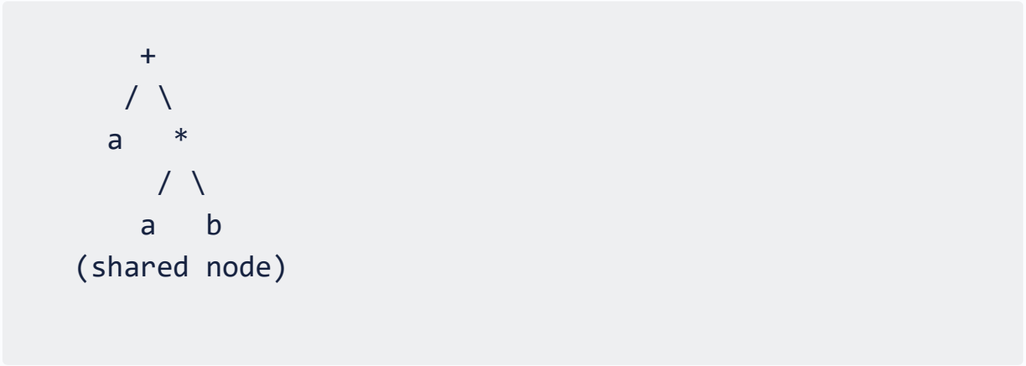
Syntax Tree — `a + b * c`

Each node represents an operator; leaves are operands. Subexpressions are **not shared**, even if identical.



DAG — Eliminating Redundancy

For `a + a * b`, the node for `a` is shared rather than duplicated, saving space and exposing common subexpressions for optimization.



DAGs are constructed by checking whether a node with the same operator and children already exists before creating a new one — a simple but powerful technique.

Key Insight

DAGs identify **common subexpressions** automatically. When the same value is computed multiple times, a DAG reuses the node — this is the foundation of **common subexpression elimination (CSE)**, a major optimization pass.

Three Address Code (TAC)

TAC is the most widely used IR form. Every instruction has **at most one operator** and uses **temporary variables** to hold intermediate results.

General Form

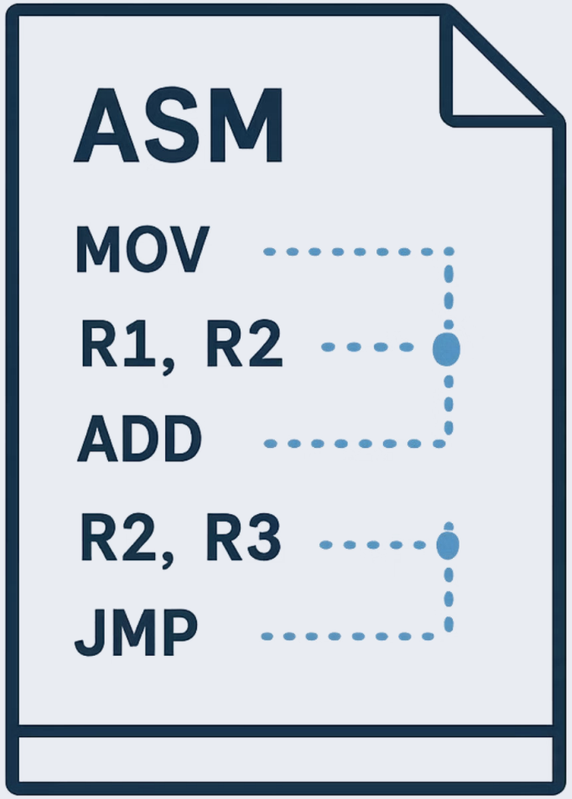
```
x = y op z
```

- `x`, `y`, `z` — variables or constants
- `op` — a binary operator (+, -, *, /)
- Unary form: `x = op y`
- Copy: `x = y`

TAC for `a + b * c`

```
t1 = b * c
t2 = a + t1
```

Temporaries like `t1`, `t2` are compiler-generated names that hold intermediate values — they map directly to registers during code generation.



Types of Three Address Statements

Assignment (Binary)

`A := B op C` — binary arithmetic or logical operation

Assignment (Unary)

`A := op B` — unary minus, plus, shift, etc.

Copy Statement

`A := B` — value of B assigned to A

Jumps

`goto L` (unconditional) or `if X relp Y goto L` (conditional)

More Statement Types

Function Calls

Sequence of `param A`, `call fun, n`, and optional `return B` statements.

Indexed Assignments

`A := B[i]` reads from offset *i* beyond B; `A[i] := B` writes to offset *i* beyond A.

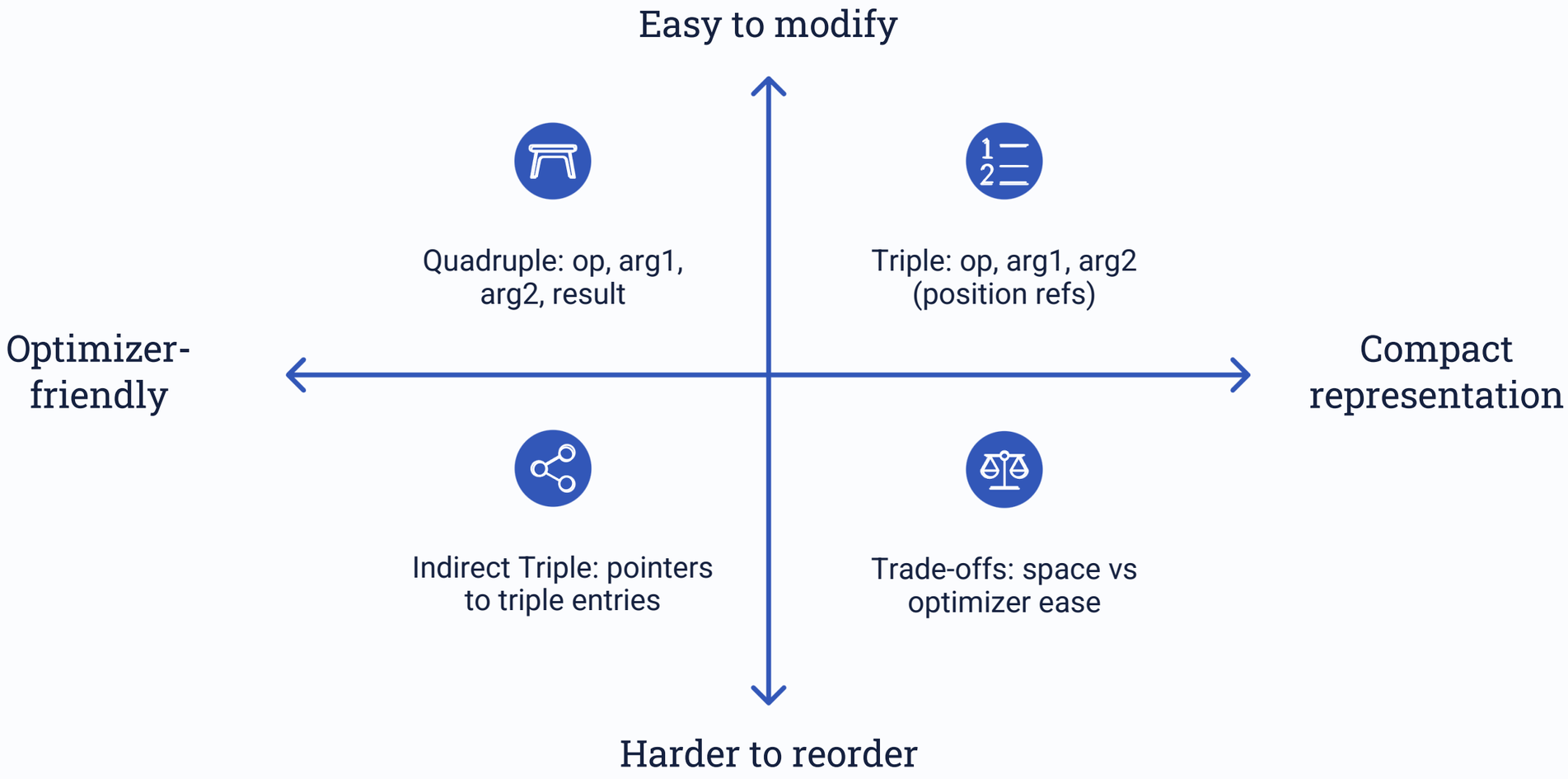
Address & Pointer

`A := &B` sets A to B's location; `A := *B` dereferences B; `*A := B` stores B at A's address.

Choice of allowable operators simplifies optimization and code generation.

Representing Three Address Code

Three address code can be stored in three structures, each with different trade-offs in space and optimizer friendliness.



The choice of representation directly impacts how easily an optimizer can rearrange statements.

Quadruple Structure

A quadruple has four fields: **operator, operand 1, operand 2, result**. Temporary variables are stored explicitly. Example for `a = b*-(c-d) + b*-(c-d)`:

Operator	Op1	Op2	Result
-	c	d	T_1
U	T_1		T_2
*	b	T_2	T_3
-	c	d	T_4
U	T_4		T_5
*	b	T_5	T_6
+	T_3	T_6	T_7
=	T_7		a

Triple & Indirect Triple

Triple

Only 3 fields — operands reference **position numbers** of prior statements instead of named temporaries. Saves symbol table space but makes reordering difficult.

No.	Op	Arg1	Arg2
(0)	-	c	d
(1)	U	(0)	
(2)	*	b	(1)
(3)	-	c	d
(4)	U	(3)	
(5)	*	b	(4)
(6)	+	(2)	(5)
(7)	=	a	(6)

Indirect Triple

A **separate pointer list** references triple entries. Statements can be reordered by changing the pointer list — no modifications to the triple structure itself.

Seq	Stmt No.
(0)	(20)
(1)	(21)
(2)	(22)
(3)	(23)
(4)	(24)
(5)	(25)
(6)	(26)
(7)	(27)

Comparing the Three Representations

Quadruple

Temporaries stored in symbol table — easy optimizer access, but wastes space.

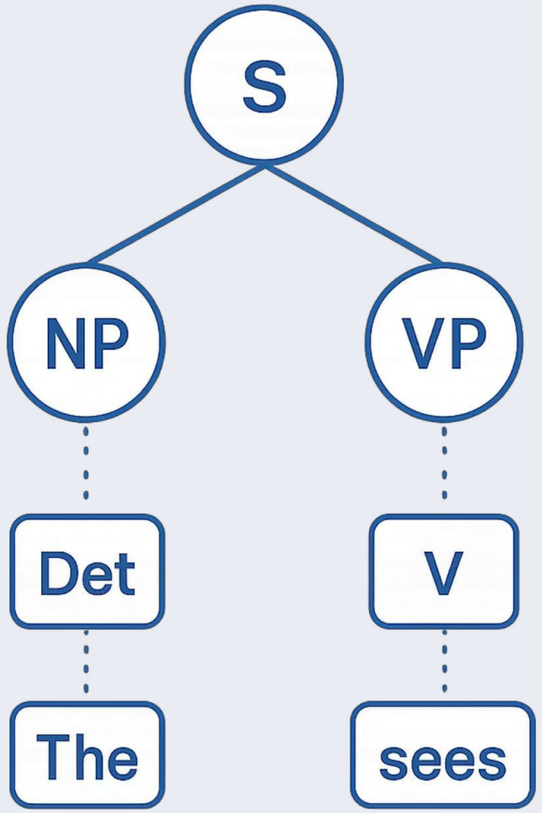
Triple

No temporaries — saves space, but moving a statement requires updating all references to it.

Indirect Triple

Reorder via pointer list only — nearly as space-efficient as quadruples; shared temporaries save extra space.

Indirect triples combine the optimizer flexibility of quadruples with the space savings of triples.



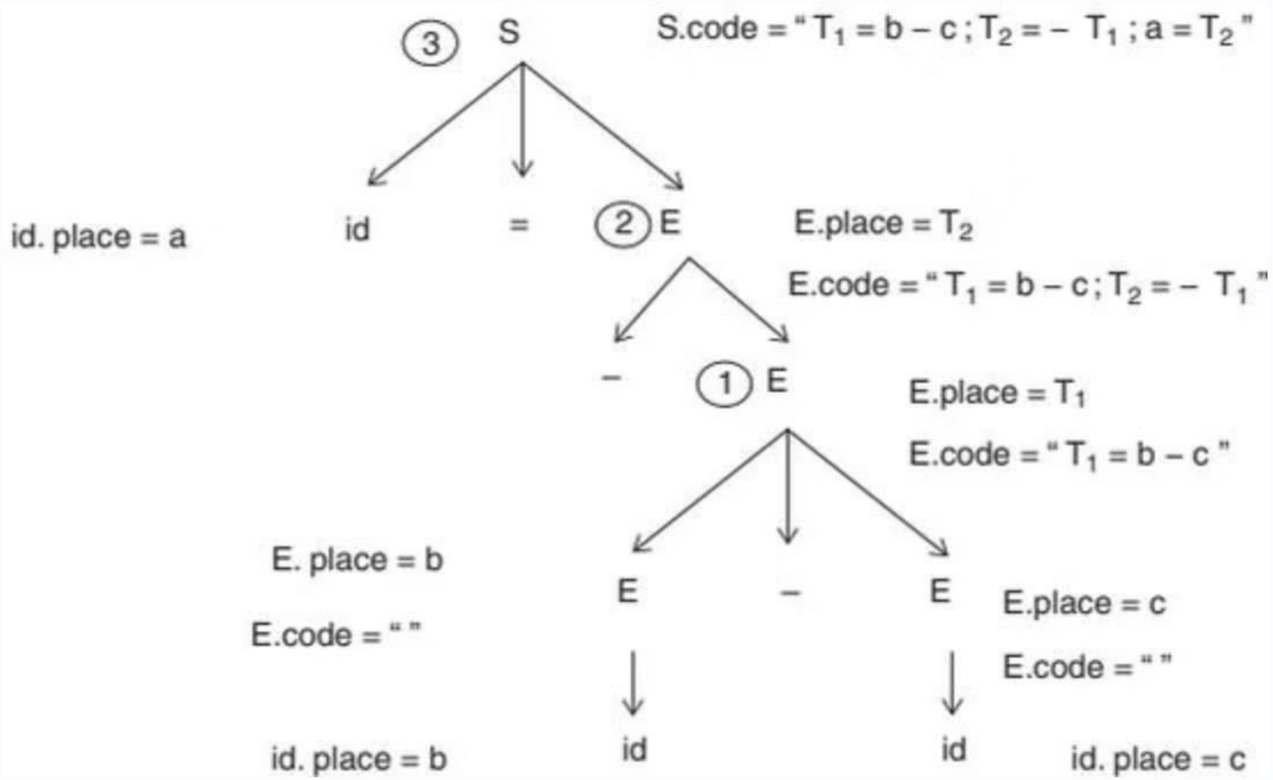
Syntax-Directed Translation

Translation rules generate three address code during parsing. Each nonterminal **E** carries two attributes:

- **E.place** — name holding the value of E
- **E.code** — sequence of three address statements evaluating E

Helper functions: `newtemp()` creates a fresh temporary; `gen()` emits a three address statement; `lookup()` searches the symbol table.

Example: `a = -(b - c)`



Generation Order

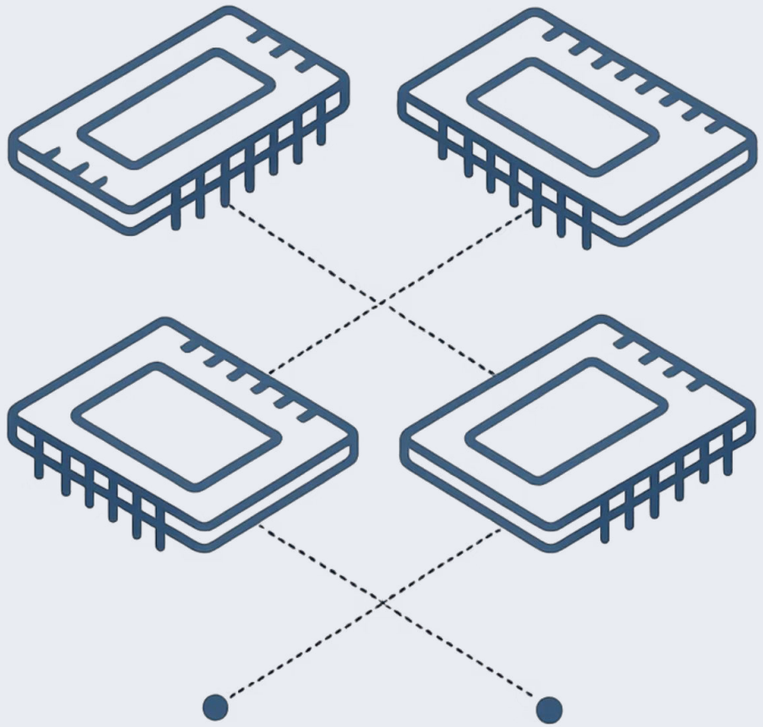
1. `E → b`, `E → c`: look up b, c in symbol table → E_1`.place=b`, E_2`.place=c`
2. $E \rightarrow E_1$ `-` E_2: create T_1 → gen T_1 `= b - c`
3. $E \rightarrow$ `-`E_1: create T_2 → gen T_2 `= -`T_1
4. `S → id = E`: look up a → gen `a =` T_2

Translation Rules for Arithmetic Statements

The syntax-directed translation for arithmetic statements can thus be written as follows:

$S \rightarrow id = E$	{ id.place = lookup(id.name); if id.place ≠ null then S.code = E.code || gen(id.place ":=" E.place) else S.code=type_error}
$E \rightarrow - E_1$	{E.place = newtemp(); E.code = E_1.code || gen(E.place ":=" "–" E_1.place)}
$E \rightarrow E_1 + E_2$	{E.place = newtemp(); E.code = E_1.code || E_2.code || gen(E.place ":=" E_1.place "+" E_2.place)}
$E \rightarrow E_1 - E_2$	{E.place = newtemp(); E.code = E_1.code || E_2.code || gen(E.place ":=" E_1.place "–" E_2.place)}
$E \rightarrow E_1 * E_2$	{E.place = newtemp(); E.code = E_1.code || E_2.code || gen(E.place ":=" E_1.place "*" E_2.place)}
$E \rightarrow E_1 / E_2$	{E.place = newtemp(); E.code = E_1.code || E_2.code || gen(E.place ":=" E_1.place "/" E_2.place)}
$E \rightarrow id$	{E.place = lookup(id.name), E.code = " "}

- These rules form the foundation of intermediate code generation for any arithmetic expression.



Addressing Array Elements

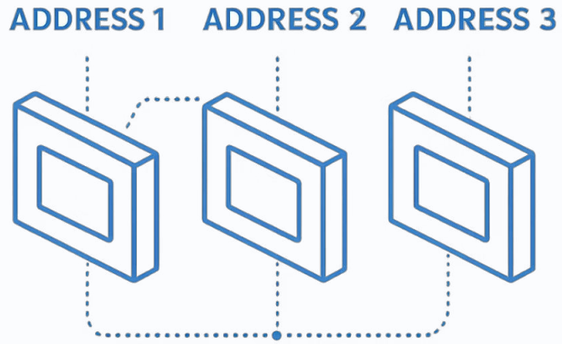
How compilers calculate memory addresses for array elements — from simple 1D arrays to multi-dimensional matrices — using base addresses, offsets, and pre-computation.

1D Array Address Formula

For a single-dimension array, the address of the **ith element** is:

$$\text{Address} = \text{base} + (i - \text{low}) \times s$$

Where **base** = address of first element, **low** = lower bound index, and **s** = size of each element.



Example: 1D Array A[5]

Array A[5] with s = 2, base address = 100, low = 1.

A[1] Address: 100	A[2] Address: 102	A[3] Address: 104
A[4] Address: 106	A[5] Address: 108	

To access A[3]: **100 + (3 − 1) × 2 = 104**

Optimizing 1D Address Computation

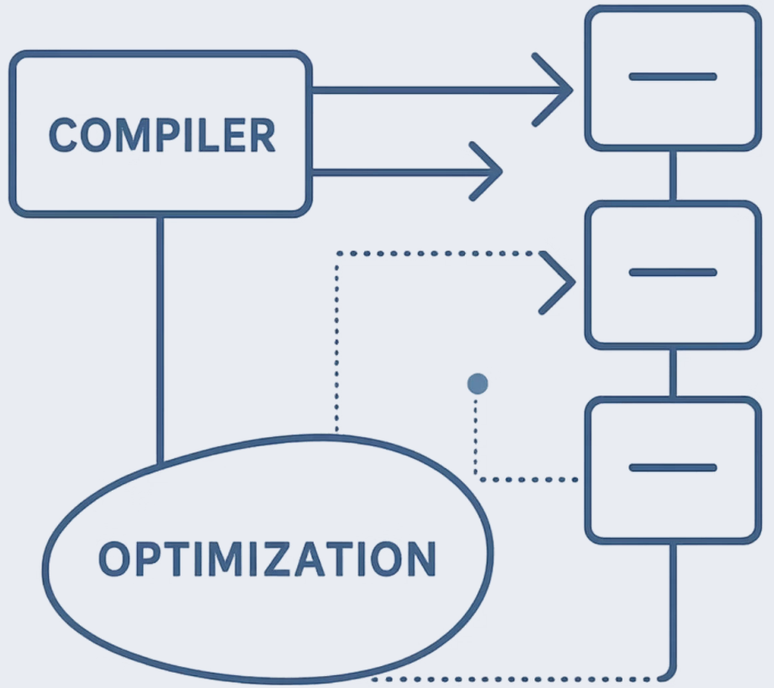
The 1D address expression can be split into two parts:

Part 1: Variable

i × s — computed at runtime based on index i.

Part 2: Pre-computed

base − low × s — all values known at compile time; stored once to speed up address generation.

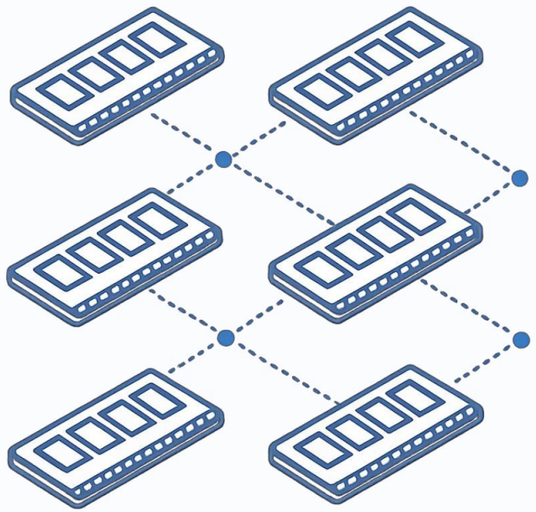


2D Arrays: Row Major vs. Column Major

Multi-dimensional arrays like matrices store elements in either **row major** or **column major** order.

Example: A[3,3] in row major order:

A[1,1]	A[1,2]	A[1,3]
A[2,1]	A[2,2]	A[2,3]
A[3,1]	A[3,2]	A[3,3]



2D Row Major Address Formula

Address of element A[i, j] in row major order:

$$\text{base} + ((i - \text{low\_i}) \times n_2 + j - \text{low\_j}) \times s$$

base

Starting address of the array

low_i, low_j

Lower bounds of each dimension

n_2

Number of columns

s

Size of each element

The second part **((i × n_j) + j) × s + (base − ((low_i × n_j) + low_j) × s)** can be pre-computed for faster address generation.

Grammar for Array Addressing

The generalized grammar rules for array element access:

```
S → A = E
E → E + E
E → E * E
E → A
A → Alist ]
A → id
Alist → Alist, E | id [ E
```

A can be either:

Simple Name

One base address, no offset (A.offset = null)

Indexed Name

Has base address + offset; assignment to a computed location

Semantic Rules: Assignment & Expressions

Code generation rules for assignments and expressions:

```
S → A = E
 If A.offset = null: gen(A.place '=' E.place)
 Else: gen(A.place '[' A.offset ']' '=' E.place)

E → E1 + E2
 E.place = newtemp()
 E.code = E1.code || E2.code ||
 gen(E.place ':=' E1.place '+' E2.place)

E → A
 If A.offset = null: E.place = A.place
 Else: E.place = newtemp()
 gen(E.place '=' A.place '[' A.offset ']')
```

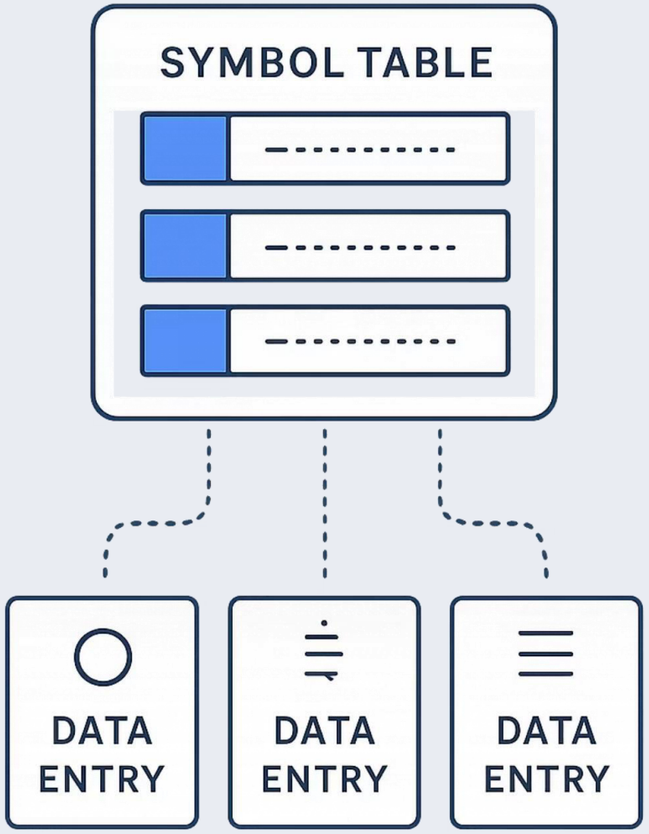
Semantic Rules: Array & Alist

```
A → Alist ]
 A.place = newtemp(); A.offset = newtemp()
 gen(A.place '=' c(Alist.array))
 gen(A.offset '=' Alist.place '+' width(Alist.array))

A → id
 A.place = id.place; A.offset = null

Alist → Alist1, E
 t = newtemp(); m = Alist1.ndim + 1
 gen(t '=' Alist.place '+' lmt(Alist.array, m))
 gen(t '=' t '+' E.place)
 Alist.array = Alist1.array; Alist.place = t; Alist.ndim = m

Alist → id [ E
 Alist.array = id.place
 Alist.place = E.place; Alist.ndim = 1
```



Key Attributes & Helper Functions

A.offset / A.place

For arrays: offset stores part 1 of expression; place stores part 2.

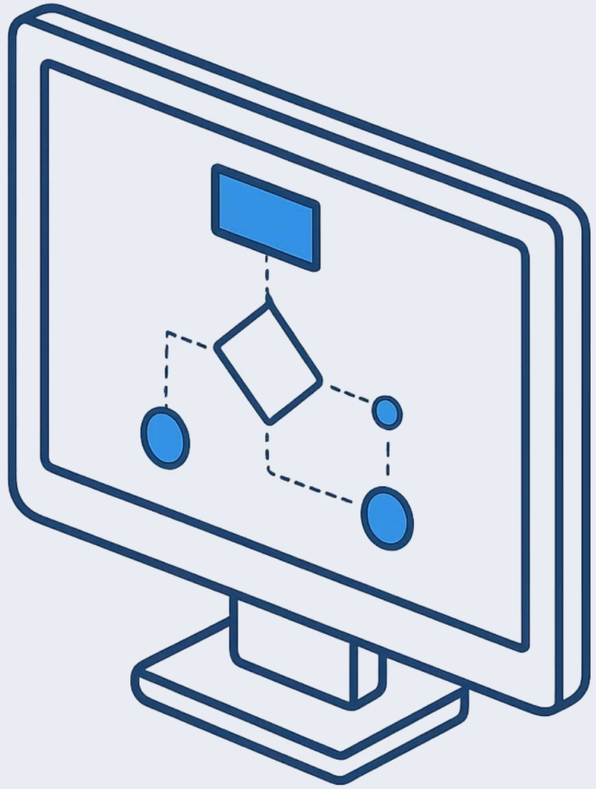
For simple variables: place points to symbol table, offset = null.

lmt() / width()

lmt() returns max elements in the j-th dimension. **width()** returns the total size of the array.

m / c()

m denotes the current array dimension. **c()** computes the second (pre-computable) component of the address expression.

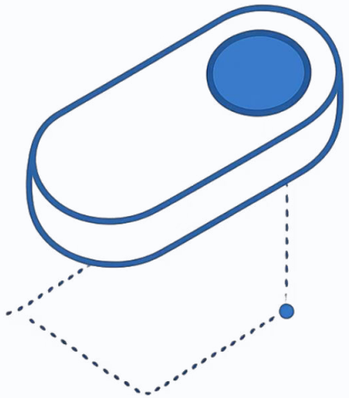


Logical Expressions & Control Statements

How compilers translate logical expressions and control flow into three-address code — the backbone of intermediate code generation.

CHAPTER 8.5

What Are Logical Expressions?



Logical expressions use relational operators (`<`, `>`, `≥`, `≤`) and logical operators (`and`, `or`, `not`) to control program flow.

> Result is always **true (1)** or **false (0)**. They determine which path of execution to follow.

Two questions must be answered after evaluation:

- Where does control go when the condition is **true**?
- Where does control go when the condition is **false**?

Grammar Rules for Logical Expressions

The production rules defining valid logical expressions:

OR / AND

NOT / Grouped

Relational

Literals

Translation Rules → Three-Address Code

$E \rightarrow E_1$ or E_2	{E.value = newtemp(); gen(E.value "=" E_1.value "or" E_2.value)}
$E \rightarrow E_1$ and E_2	{E.value = newtemp(); gen(E.value "=" E_1.value "and" E_2.value)}
$E \rightarrow$ not E_1	{E.value = newtemp(); gen(E.value "=" "not" E_1.value)}
$E \rightarrow (E_1)$	{E.value = E_1.value}
$E \rightarrow id_1$ **relop** id_2	{E.value = newtemp(); gen("if" id_1.value relop.op id_2.value "goto" nextstat + 3) gen(E.value "=" "0") gen("goto" nextstat + 2) gen(E.value "=" "1")}
$E \rightarrow$ true	{E.value = newtemp(); gen(E.value "=" "1")}
$E \rightarrow$ false	{E.value = newtemp(); gen(E.value "=" "0")}

- For relational expressions, a conditional `goto` is generated, jumping ahead by offsets to assign 0 or 1 using `nextstat`.

Worked Examples: Logical Expressions

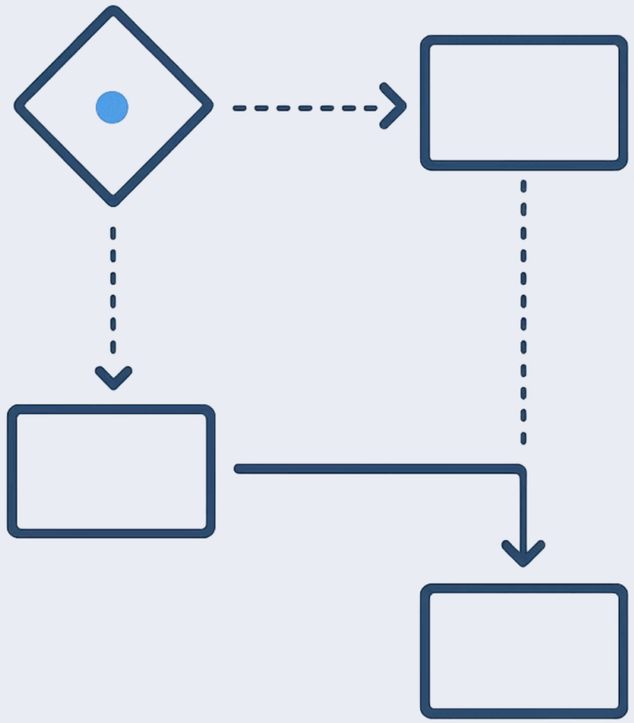
Example 4 — `x or y and not z`

```
t₁ = not z
t₂ = y and t₁
t₃ = x or t₂
```

Example 5 — `if x < y then 1 else 0`

```
1. if x < y goto 4
2. t₁ = 0
3. goto (next)
4. t₁ = 1
```

The relational check generates a conditional jump; the false branch assigns 0, the true branch assigns 1.



Control Flow Statements

Control statements direct execution based on conditions. The three core constructs are:

if E then S_1

Conditional — executes S_1 only when E is true.

if E then S_1 else S_2

Branching — executes S_1 on true, S_2 on false.

while E do S_1

Loop — repeats S_1 as long as E remains true.

Control Flow Diagrams

Figure 8.9 — if-then-else

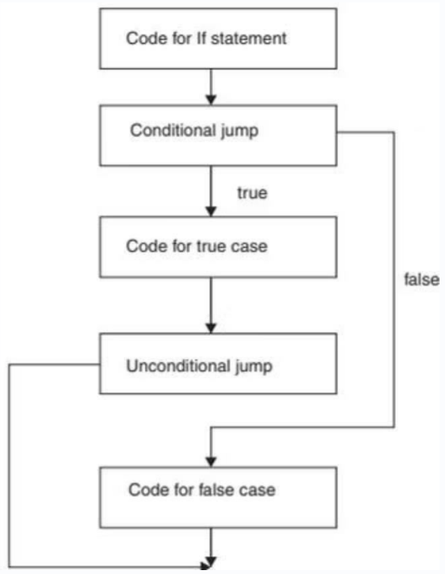
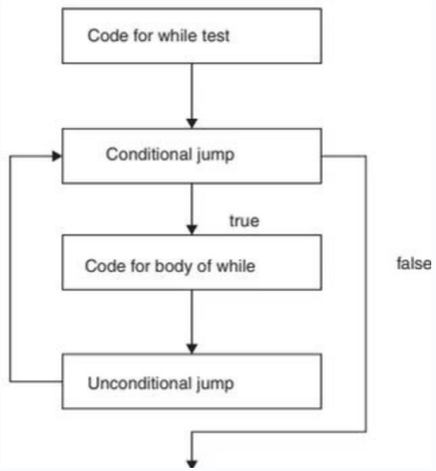


Figure 8.10 — while loop



$S \rightarrow$ if E then S_1	{E.true = newlabel(); E.false = S.next; S_1.next = S.next; S.code = E.code || gen(E.true , " :") || S_1.code}
$S \rightarrow$ if E then S_1 else S_2	{E.true = newlabel(); E.false = newlabel(); S_1.next = S.next; S_2.next = S.next; S.code = E.code || gen(E.true , " :") || S_1.code || gen(" GOTO ", S.next) || gen(E.false , " :") || S_2.code}
$S \rightarrow$ while E do S_1	{S.begin = newlabel(); E.true = newlabel(); E.false = S.next; S_1.next = S.next; S.code = gen(S.begin ":") || E.code || gen(E.true , " :") || S_1.code || gen("GOTO" , S.begin)}

Labels are generated for true/false branches. The `while` loop uses a **begin label** to loop back, and a **false label** to exit. The `if-then-else` uses a `GOTO` to skip the false branch after the true branch executes.

Worked Examples: Control Statements

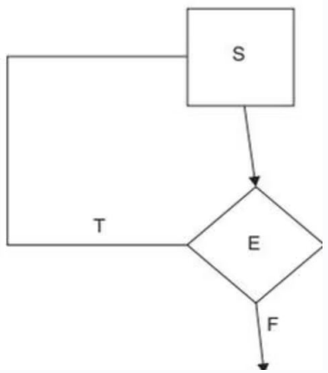
Example 6 — `while (a < 5) do a := b + 2`

```
L1: if a < 5 goto L2
    goto last
L2: t1 = b + 2
    a = t1
    goto L1
last:
```

Example 7 — `do x = y + z while (a < 10)`

```
L1: t₁ = y + z
    x = t₁
    if a < 10 goto L1
```

In `do...while`, the body executes **first**, then the condition is checked — the loop back is conditional.



For Loop Control Flow

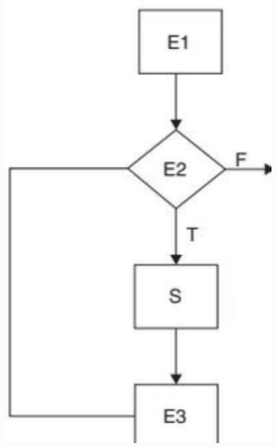
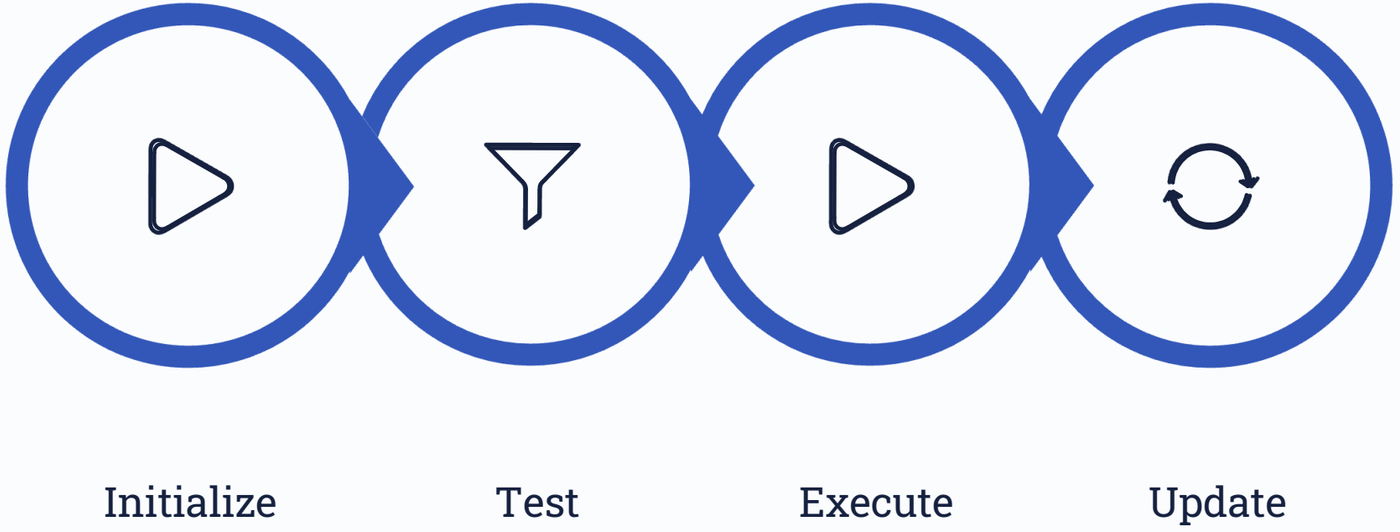


Figure 8.12 — `for(E1; E2; E3) S`



E1 runs once. E2 is tested each iteration — false exits the loop. S executes on true, then E3 updates before looping back.

Switch Statement Translation

The `switch(E)` statement evaluates E once, then jumps to the matching case via a **test block**:

```
t1 = E
goto test
L1: s1; goto last
L2: s2; goto last
...
Ln: sn; goto last
test:
  if (t1==v1) goto L1
  if (t1==v2) goto L2
  ...
  if (t1==vn) goto Ln
last:
```

Key insight: All case bodies jump to `last` after execution. The test block at the end compares the stored value against each case label and dispatches accordingly.

Evaluate once

Store E in a temp `t1` before branching.

Centralized test

All comparisons happen in a single test block.

Fall-through prevention

Each case ends with `goto last`.